

Assignment 1

Digital Signal Processing

Fourier Transform

University of Glasgow, School of Engineering
Bernd Porr, Georgina Klemencic & Scott Watson

This assignment is about the **Fourier Transform**.

Form groups of two, work together and submit **one** report. Enter your team into the Wiki on Moodle, listing your names and matriculation numbers. Use the Moodle forum to find teammates.

Record the voice of one member of your group as two uncompressed WAV files at a distance of:

- 5cm
- 1m

Say one complete sentence. It can be any language. Make sure that you record with a sampling rate of at least 44 kHz or higher and that the audio is not clipped. The full audio spectrum up to 20 kHz needs to be available. Reports based on recordings at sampling rates below 44 kHz or low-quality downloads will not be marked. For both recordings, do the following:

1. Load the audio samples into Python.
 - i. Plot the voice audio signal in the time domain (linear axis: normalised amplitudes vs time) and
 - ii. in the frequency domain. Use a logarithmic axis for both the frequency and amplitude, where the amplitude should be in dB with proper axis labels.
 - iii. Identify visually in the frequency plot the fundamental frequencies of the different vowels spoken. Explain in the report why you have chosen these peaks. Do not use automatic detection. Instead, use your knowledge of fundamentals and human vowels.
 - iv. Mark the frequency range which contains the consonants up to the highest frequencies containing them. Again, do not use an automated process, but inspect the spectrum and explain the chosen range.
 - v. Mark the region which most likely contains just noise. Again, explain your reasoning.

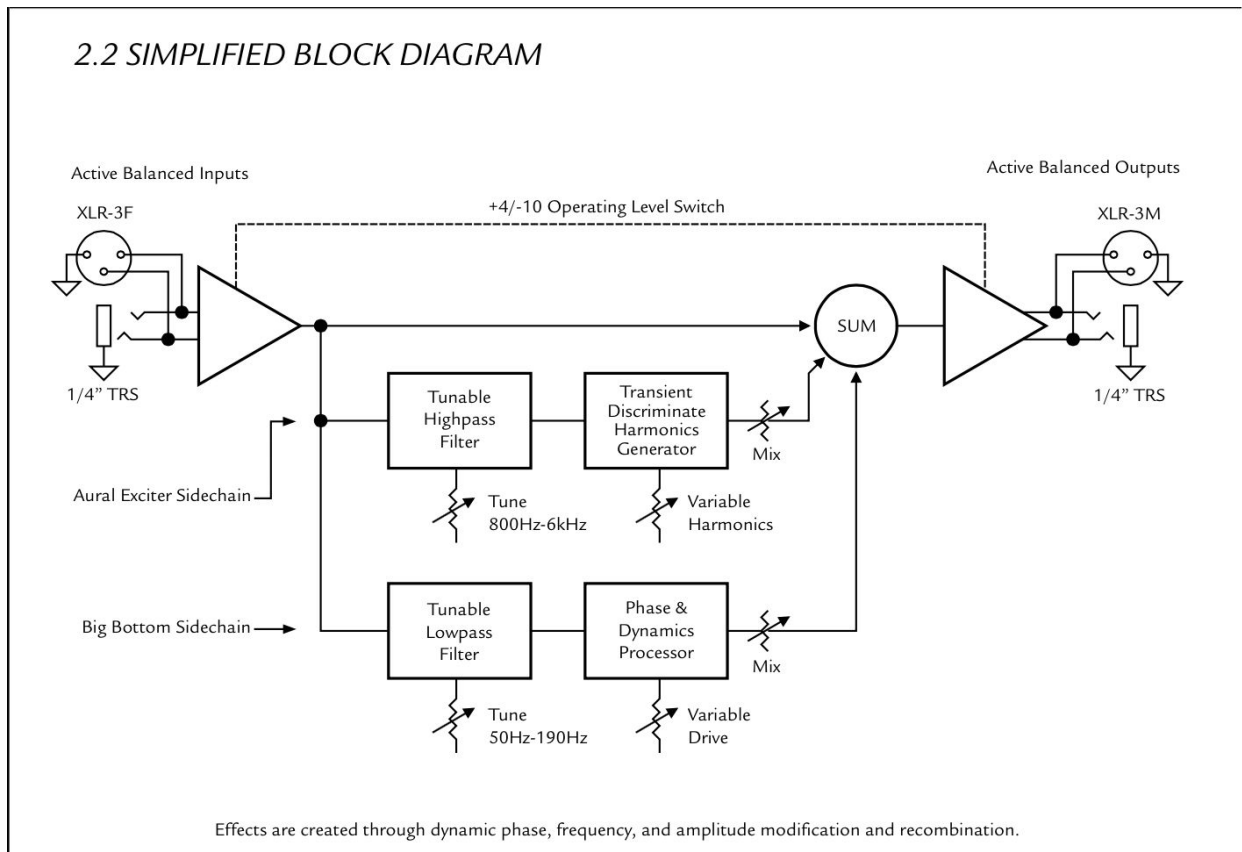
20%

2. Enhance both voice recordings so that a) the voice sounds clearer and more interesting as heard on radio shows, and b) remove noise. Use the FFT to translate the voice recording from the time domain into a

frequency spectrum, then manipulate the amplitudes of the frequency coefficients by either boosting or reducing them. Finally, do an IFFT and transfer them back into the time domain. For both recordings, explain what kind of noise needs to be removed and which frequency range it has. Also, for both recordings, explain how you would like to improve the speech by looking into the loss of bass frequencies with distance, and the risk of pop sounds when the microphone is close.

20%

3. The **aural exciter** is a widely used device in broadcast to make a voice sound more prominent and interesting. It is used heavily by private radio stations and in TV commercials. The actual amplitude of the voice is not changed much, but the voice of the presenter is perceived as louder. The idea is that if one sends audio through a non-linearity (tanh, etc), harmonics are created. These harmonics excite the human ear at 6 kHz where it is most sensitive. However, not all harmonics are pleasant, so the exciter feeds only a certain frequency band into the non-linearity and then also only picks certain harmonics. Below is the block diagram of the Aphex Aural Exciter:



Use your recording at close distance to the microphone and then create your own exciter which enhances your voice. Feel free to remove any pop sounds beforehand by manipulating the FFT coefficients as in Task 2. Show the iterative development of your exciter in the report. Start with an initial design, examine the result, and use what you observe to decide what to change. Use at least two annotated graphs to show the effect of your changes and explain why you chose your final design.

60%

Further instructions:

The report should be brief, concentrating on the technical aspects and why you have done the different steps. Do not add generic theory about voice or Fourier Transform. Just describe the method and the result. All figures in the report must be high-quality graphics in vector format. Blurry JPEG figures or screenshots will not be marked. Submissions must be PDF.

Complete Python code must be included in the appendix and submitted via Moodle alongside the report. Upload your code and data/WAV files to Moodle in the form of a single zip file. Follow the exact naming conventions for all files as specified on Moodle. The scripts will be tested under Linux from the command line (not in Spyder, PyCharm, or VS Code).

Make sure your code is:

- platform-independent
- does not contain absolute paths
- Code that crashes will result in low marks. The same applies to code which does not display any plots.
- No high-level Python signal processing/filtering commands are allowed (including, for example, `fftfreq`) except for the FFT and IFFT commands. Any use of high-level code will result in zero marks for that part.
- Do not use FIR/IIR filtering in this assignment. The purpose of this assignment is to assess your use of the FFT/IFFT, so filtering must be carried out by manipulating the FFT coefficients directly. Any use of FIR/IIR filtering will result in zero marks for that part.
- The code needs to be efficient. For example, calculating 1M frequency samples and then only using one, discarding the rest, will result in lower marks.
- Inflated/unreadable code generated by an LLM will result in low or zero marks. It is important that a human can understand the code generated. All tasks above can be solved elegantly with very few lines of code. If you use GenAI, then take its raw output and optimise it.
- GenAI tools may be used in this assignment. If you use GenAI, you must include a short acknowledgement in your report stating which tool(s) you used (name, version, publisher), how you used them, and what work you contributed yourself. See the [UoG guidance](#) on acknowledging GenAI use.
- We reserve the right to interview you to test if you own the code (i.e. that you can explain it).

Deadline: Monday 19th October, 3pm on Moodle.